

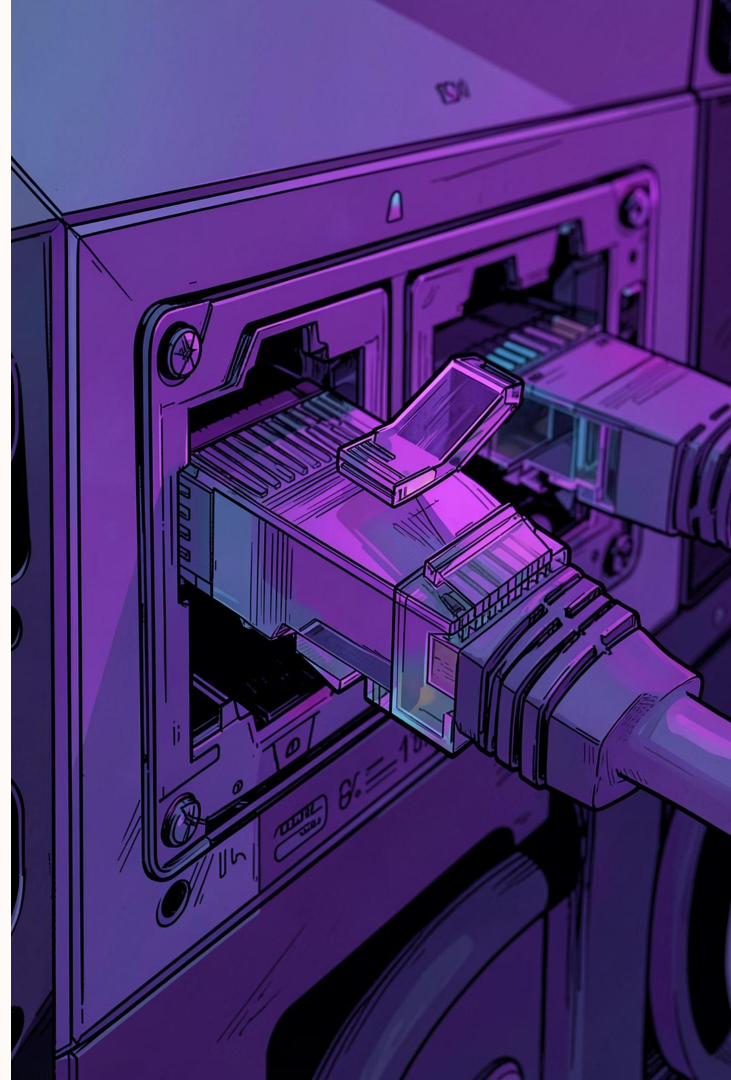
tasks 1: Basic Network Sniffer

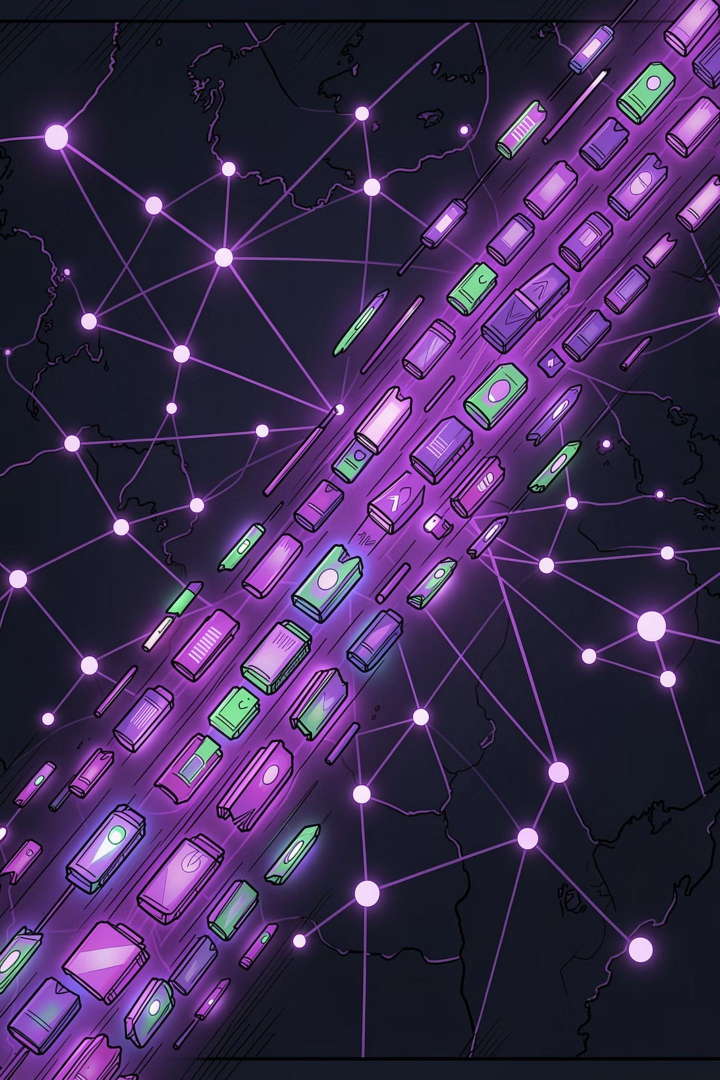
```
from scapy.all import *
```

```
def packet_callback(packet):
```

```
    if packet.haslayer(IP):  
  
        print("\n----- Packet Captured -----")  
  
        print("Source IP:",  
              packet[IP].src)  
  
        print("Destination IP:",  
              packet[IP].dst)  
  
        print("Protocol:",  
              packet[IP].proto)
```

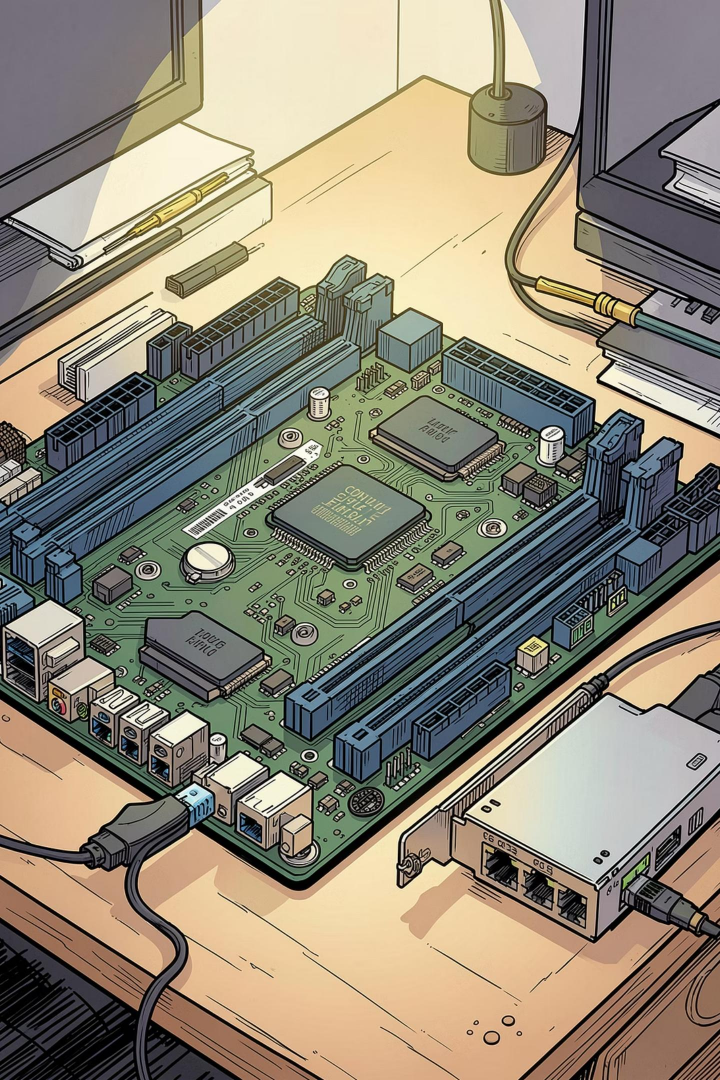
```
sniff(prn=packet_callback, store=False)
```





The Digital Tapestry

- Every click, stream, and email is a sequence of packets crossing links.
- Your machine constantly exchanges packets — sniffers let you observe that conversation.
- **Ethical reminder:** Only sniff on networks you own or a controlled lab; respect privacy and law.



How Packets Move

01

NIC (Network Interface Card)

Receives frames from the physical medium and delivers them up the stack.

02

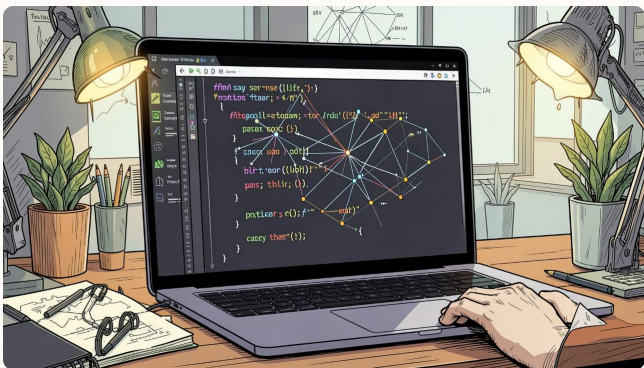
Normal Mode

NIC filters frames not addressed to the host—you only see your traffic.

03

Promiscuous / Monitor Mode

NIC forwards all frames to the OS so the sniffer can inspect every packet on the link.



Why Python + Scapy?

Python provides rapid prototyping and readable code. Scapy treats packets as native Python objects for building, sending, capturing, and dissecting packets with minimal boilerplate.

Rapid

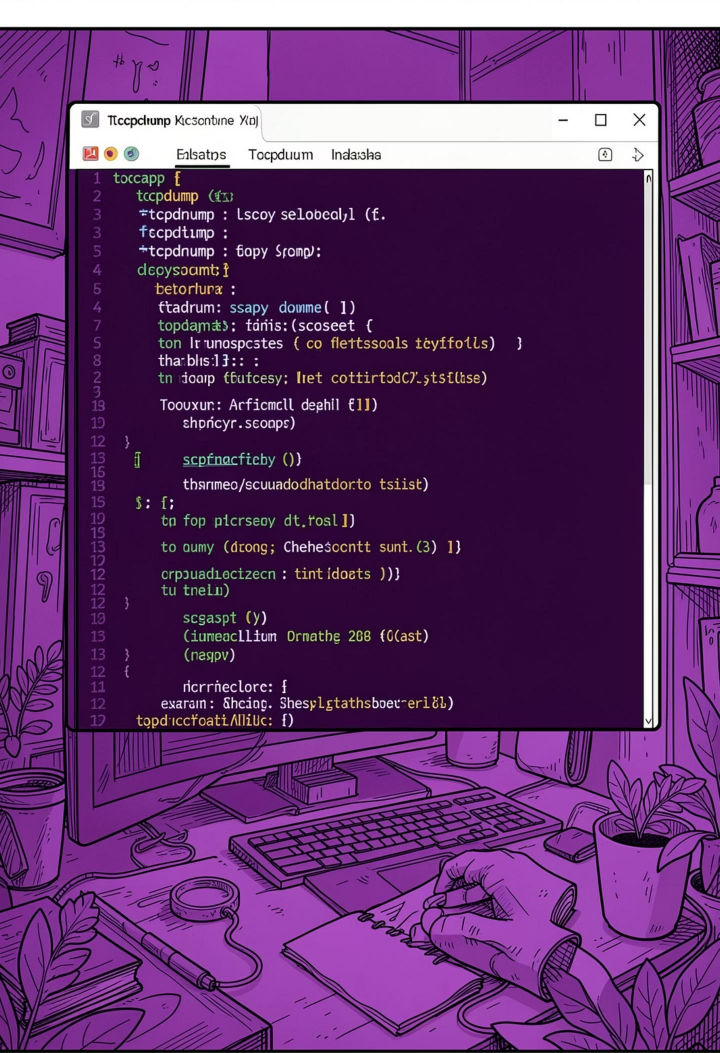
Fast to write and iterate network tools.

Powerful

Parse layers (Ether, IP, TCP/UDP) and payloads easily.

Testable

Craft packets to test defenses and simulate traffic.



The Core Toolset

Key building blocks you'll use:

- **sniff()** — capture packets in real time (Scapy).
- **BPF filters** — narrow capture (e.g., "tcp and port 80").
- **Callbacks** — process each packet as it arrives for logging or analysis.



Writing the Logic

Structure your sniffer around a short, clear callback that inspects layers and extracts metadata.

Step 1 — Capture

```
sniff(iface='eth0', prn=callback, store=0,  
filter='tcp')
```

Step 2 — Inspect

Check for IP/TCP/UDP layers before accessing fields to avoid exceptions.

Step 3 — Extract

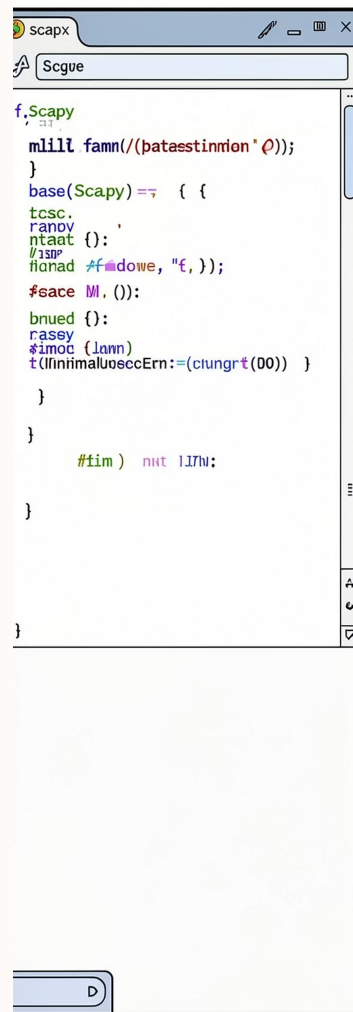
Get src/dst IPs, ports, protocol, flags, and payload bytes for logging or analysis.

Minimal Scapy Example

Concise, copy-ready snippet to run in a lab (run as root):

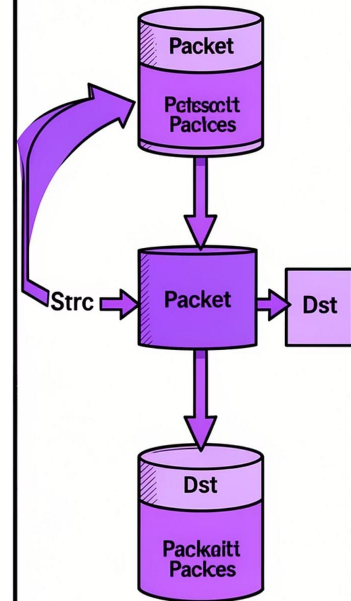
```
from scapy.all import sniff, IP, TCP

def callback(pkt):
    if IP in pkt:
        src = pkt[IP].src
        dst = pkt[IP].dst
        proto = pkt[IP].proto
        print(f"{src} -> {dst} proto={proto}")
    if TCP in pkt:
        sport = pkt[TCP].sport
        dport = pkt[TCP].dport
        flags = pkt[TCP].flags
        print(f"TCP {sport} -> {dport} flags={flags}")
    sniff(prn=callback, store=0)
```



The image shows a screenshot of a web-based code editor titled 'Scgue' with a tab labeled 'scapx'. The editor contains a Python script for Scapy. The script defines a function 'f, Scapy' (likely a typo for 'def callback') that uses 'sniff' to capture packets. It checks for IP and TCP layers in the packet and prints their details. The script is incomplete, with several lines of code appearing to be garbled or placeholder text.

```
f, Scapy
mllll famn((patesstinmion' e));
}
base(Scapy) == { {
tcsc.
ranov
ntaat {}: '
//130P
flonad #f#dowe, "f, ));
#eace M, ());
brued {}:
rassy
simoc {lann)
t(lfminimaluocccErn:=(ciungtrt(00)) }
}
}
#tim) nnt l17N:
}
}
}
```



Inspecting Payloads & Protocols

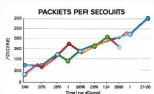
What to look for when analyzing packet contents:

- Application data (HTTP headers, cookies, forms) — often plaintext if not TLS.
- TCP flags (SYN/ACK/FIN/RST) reveal connection state transitions.
- UDP packets often carry DNS, DHCP, or streaming payloads — inspect payload length and structure.
- **Security note:** Unencrypted credentials and sensitive data can appear — never exfiltrate or misuse.



```
"oiitpr==(leauahs)">{
  /Tcaesvesnuirps/itc{
    htcheP:-----
    http="WU6LHD qmrlf codintinde" (/t,"n,peyaiclnke")
    HHTrp=/rtDhl.yenfcasvertmi { (/t,"n,peyaiclnke")
    hhttp=Enegshiup=Clakques ()
    wflarrldoade". vnanantenbus, {
      //llackWlaafde-enImnos: boahd'licm"-Flat"safchuh",)
      dirft/imaped_gimno,""g Plaitáarl")}{
      wftnkl aairm onoeck:: unenon!;"
      luitctnlorhfee=Lstean(),){
      tttlhrasrtine= fcpaltciul=). {
      wltloallueaiory chanat;oadn (0))      'ry) '{
      witcéatllxiart dhtáblcl {
      owhtbo="{ bptiane"ththe","/pam(""/h,pfatchnaP")
      tlV/startur thinn(ba ainchtod)
      orebt trmonefniórah{"ouuraínnecs",{fls; ::ntorraie,oFls=puy'i)
      tytob hñse", "esgwoonlo: fnlalaigoc"}
      lppot unidld; fegwvoni{"
      jmtutúdy: bdesngesabtc,()"} }
      uifpouro. irakreeSlcatintions(fifchex fyll":gbnps"a))
```


Beyond the Basics — Analysis & Visualization



Traffic Over Time

Plot packet rates (Matplotlib) to find spikes and anomalies.



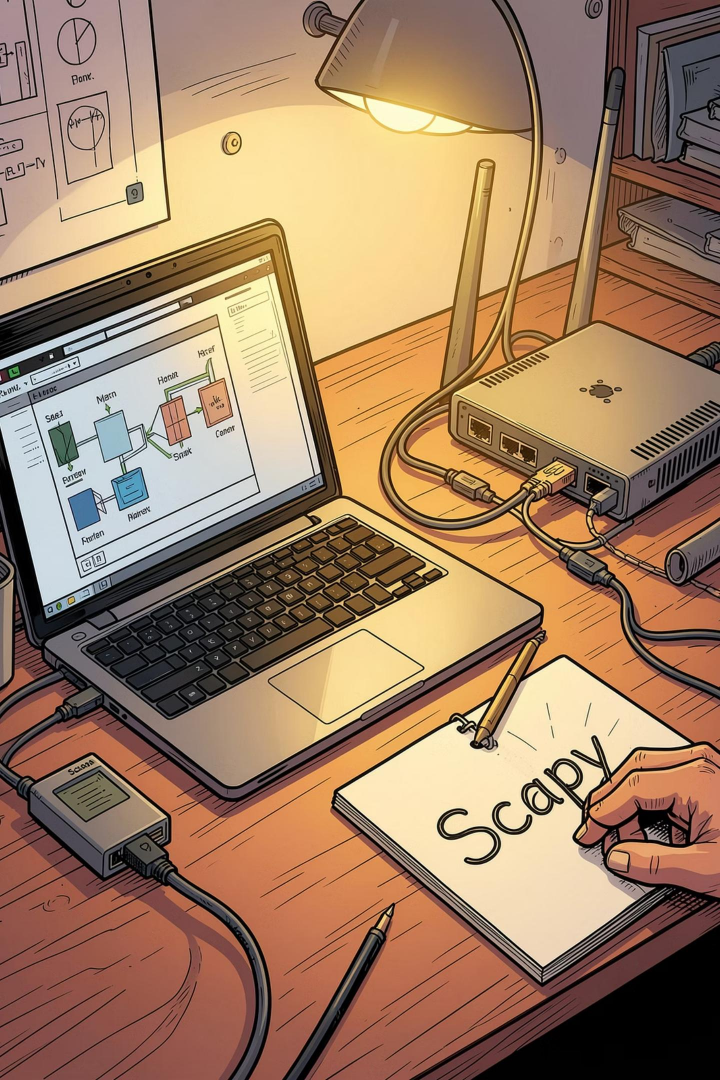
Connection Graphs

Visualize top talkers and flows to identify hubs or suspicious behavior.



Anomaly Detection

Use ML (clustering, isolation forests) on features like packet sizes, interarrival times, and flags.



Your Turn: Safe, Practical Next Steps

1. Install Scapy: `pip install scapy` (or use your distro's package manager).
2. Identify interface: use `ip addr` / `ifconfig`; run sniff with appropriate permissions.
3. Start small: capture limited traffic with BPF filters (e.g., "tcp port 80").
4. Log responsibly: store captures locally, mask or discard sensitive payloads.
5. Iterate: add parsing, visualization, and anomaly detection as your toolkit grows.

❏ Need a ready-to-run repo or a GUI prototype? Ask and I'll produce a focused script or a starter project with visualization hooks.